

Guión de Presentación — Agente RAG Retail S.A.

IE1 — Diseño del proyecto de agente IA (15%)

Qué decir:

El proyecto nació de un problema real: el equipo de adquisiciones de Retail S.A. tomaba decisiones de reabastecimiento de forma manual, sin cruzar variables externas como el clima o eventos comerciales. Eso generaba quiebres de stock en temporada alta y sobrestock en productos de baja rotación.

La solución que diseñamos es un agente de inteligencia artificial basado en arquitectura RAG. No reemplaza al encargado de compras, sino que le entrega información procesada y fundamentada para que decida más rápido y con mejor respaldo.

El agente cubre tres flujos operativos concretos: - Consulta libre en lenguaje natural sobre el estado del inventario - Análisis de alerta por SKU, con estado crítico, alerta o normal - Generación de recomendación de orden de compra con cantidad, proveedor, fecha y costo estimado

Cada uno de estos flujos responde a una necesidad específica del equipo: la consulta libre para el jefe de adquisiciones, la alerta para bodega, y el pedido para compras y finanzas.

Qué mostrar: La pantalla del frontend con las tres secciones. Explicar brevemente qué hace cada una antes de hacer cualquier demo.

IE2 — Elaboración de prompts (10%)

Qué decir:

Los prompts son el núcleo del comportamiento del agente. Diseñamos tres prompts de sistema distintos, uno por endpoint, y cada uno tiene decisiones técnicas deliberadas.

El prompt de consulta le dice al modelo que responda solo con la información del contexto recuperado, y que si no tiene datos suficientes lo diga explícitamente. Eso evita que el modelo invente información sobre stock o precios.

El prompt de alerta obliga al modelo a responder en un formato fijo: ESTADO, ANÁLISIS, ACCIÓN, FUENTE. Eso no es solo estética: cuando la respuesta tiene etiquetas conocidas,

puede ser parseada automáticamente por otros sistemas. Es una decisión de arquitectura disfrazada de prompt.

El prompt de pedido va más lejos. Codifica directamente la política interna de la empresa: demanda proyectada a 30 días, más 20% de buffer, menos stock actual. Y marca automáticamente si el pedido requiere aprobación gerencial por superar los cinco millones de pesos. Esto garantiza que la regla de negocio se aplique siempre igual, independiente de cómo esté redactada la pregunta.

Qué mostrar: Abrir `main.py` y mostrar el bloque content del system prompt de cada endpoint. Señalar las líneas 74-79, 109-115 y 149-157.

IE3 — Flujos RAG (10%)

Qué decir:

El sistema tiene dos tipos de fuentes de datos. Las internas son todo lo que ya conoce la empresa: el catálogo de productos con SKU, proveedor, lead time y costo unitario; el inventario actual con stock real y mínimos; y el historial de ventas de los últimos tres meses con clasificación de tendencia. Las externas son el pronóstico climático para Los Lagos, el calendario de eventos comerciales como Cyber Monday y Navidad, y las tendencias de búsqueda de mercado.

Toda esa información vive en un solo archivo: `data/inventario.txt`. Al iniciar el sistema, ese archivo se divide en fragmentos de 500 caracteres con un overlap de 50, se convierte en vectores de embedding usando el modelo `text-embedding-3-small`, y se almacena en ChromaDB de forma persistente en disco.

Cuando llega una consulta, el sistema la convierte en un vector y busca los cuatro fragmentos más similares en la base de vectores. Solo esos fragmentos se le pasan al modelo como contexto. El modelo no ve el archivo completo, no tiene acceso a internet, y no puede usar conocimiento externo a lo que se le entrega.

Qué mostrar: Abrir `rag.py` completo. Señalar el `chunk_size=500`, el `chunk_overlap=50`, y el `k=4` en la última línea. Luego abrir `data/inventario.txt` y mostrar que tiene secciones de clima, eventos y tendencias de mercado además del inventario.

IE4 — Coherencia entre datos y respuestas (10%)

Qué decir:

Para que el sistema sea útil en un contexto de negocio real, necesita ser confiable. Implementamos tres mecanismos para garantizar que las respuestas sean coherentes con los datos.

Primero, temperatura 0.1 en todos los endpoints. Cuanto más baja la temperatura, más determinista es el modelo. Dos consultas idénticas producen respuestas prácticamente idénticas. Para decisiones de compra, eso es fundamental: si el mismo SKU con el mismo stock genera recomendaciones distintas cada vez, el usuario pierde confianza en la herramienta.

Segundo, la instrucción explícita de no suponer. Si los datos recuperados no responden la pregunta, el modelo lo dice. Lo verificamos con preguntas sobre productos que no existen en el catálogo: el sistema responde que no tiene esa información, no inventa nada.

Tercero, el campo FUENTE en las respuestas de alerta. El modelo debe indicar qué datos del contexto usó para llegar a su conclusión. Eso permite al usuario verificar si la recomendación está basada en información actualizada o no.

Qué mostrar: Hacer una demo en vivo. Preguntar por ELEC-001 en el endpoint de alerta y mostrar que la respuesta menciona stock 8 vs mínimo 15, tendencia creciente, y que cita la fuente. Después preguntar por un SKU inventado y mostrar que el sistema reconoce que no tiene esa información.

IE5 — Arquitectura del sistema (15%)

Qué decir:

La arquitectura se divide en tres módulos funcionales que operan en secuencia.

El módulo de recuperación toma la consulta del usuario, la convierte en un vector de embedding, y busca en ChromaDB los cuatro fragmentos más relevantes por similitud semántica. Esto es lo que le da al sistema la capacidad de entender preguntas en lenguaje natural sin necesidad de palabras clave exactas.

El módulo de procesamiento ensambla el prompt final. Toma el system prompt del endpoint correspondiente, le agrega el contexto recuperado y la pregunta original, y construye el

mensaje que le va al modelo. Aquí también se aplican las reglas de negocio, que están embebidas en el system prompt.

El módulo de generación llama a GPT-4o-mini a través de la API de GitHub Models, con temperatura 0.1, y retorna la respuesta estructurada al cliente.

Sobre eso, hay una capa de API en FastAPI con tres endpoints REST, y una interfaz web en HTML5 puro que consume esos endpoints.

La decisión de usar GPT-4o-mini en lugar de modelos más grandes fue deliberada: para tareas de síntesis de información acotada, el rendimiento es equivalente a una fracción del costo. En producción, eso importa.

Qué mostrar: El diagrama de arquitectura del informe técnico. Describir el flujo de arriba a abajo mientras se señalan los componentes.

IE6 — Diagrama de arquitectura (10%)

Qué decir:

El diagrama muestra el flujo completo desde que el usuario escribe una pregunta hasta que recibe una respuesta. Cada bloque representa un módulo funcional real del sistema, no una abstracción teórica.

Se puede leer de arriba hacia abajo: el usuario interactúa con el frontend, que hace un POST a uno de los tres endpoints de FastAPI. FastAPI llama al retriever, que va a ChromaDB y trae los fragmentos relevantes. Esos fragmentos se combinan con el prompt de sistema y la consulta original, y el conjunto se envía a GPT-4o-mini. La respuesta vuelve al usuario en formato estructurado.

Lo que vale la pena destacar es que ChromaDB y el archivo de inventario están en la misma capa de base de conocimiento. ChromaDB es la representación vectorial de ese archivo, construida una sola vez y reutilizada en cada consulta.

Qué mostrar: El diagrama del informe técnico (sección 6). Si es presentación en pantalla, hacer zoom en el bloque de la base de conocimiento para explicar la relación entre el archivo de texto y el vector store.

IE7 — Fundamentación de decisiones de diseño (10%)

Qué decir:

Cada decisión técnica del proyecto tiene una justificación de negocio.

RAG en lugar de fine-tuning: el inventario cambia. Los precios, los stocks, los proveedores se actualizan constantemente. Con RAG, basta editar el archivo de texto. Con fine-tuning habría que reentrenar el modelo cada vez. Para Retail S.A. eso es inviable operacionalmente.

ChromaDB local en lugar de un servicio en la nube: la base de conocimiento es pequeña, seis SKUs, menos de tres kilobytes. Un servicio externo añadiría latencia y costo sin ningún beneficio real a esta escala. Si el catálogo crece significativamente, esa decisión se revisaría.

Políticas de negocio en el prompt en lugar del código: el margen del 20% y el umbral de aprobación de cinco millones están escritos en el system prompt, no en el código Python. Eso significa que el equipo de negocios puede ajustarlos sin tocar el backend.

Tres endpoints especializados en lugar de uno genérico: cada flujo tiene un output con estructura distinta y va a usuarios distintos. Separarlos permite aplicar prompts específicos y eventualmente conectar cada endpoint a sistemas diferentes, como el ERP, el correo de aprobaciones o el sistema de bodega.

Qué mostrar: Mostrar el system prompt del endpoint de pedido en `main.py` (líneas 149-157) y señalar dónde están codificadas las reglas. Luego mostrar que en `rag.py` no hay ninguna lógica de negocio, está completamente separada.

IE8 — Informe técnico (10%)

Qué decir:

El informe técnico consolida todo el diseño en un documento estructurado. Incluye las especificaciones técnicas del sistema en tabla, el catálogo de productos con estado de stock, la red de proveedores con lead times y descuentos, los prompts completos con justificación de cada decisión, el diagrama de arquitectura, y los resultados de validación con escenarios concretos probados.

La sección de validación es importante: no solo describimos el sistema, probamos cuatro escenarios reales y documentamos los resultados. Stock crítico detectado correctamente,

alerta con tendencia incorporada, pedido con flag de aprobación activado, y consulta fuera del catálogo respondida sin alucinación.

Qué mostrar: Abrir el PDF del informe técnico. Navegar rápidamente por las secciones mostrando las tablas y el diagrama. Detenerse en la sección 8.5 de validación.

IE9 — Lenguaje técnico y evidencias (10%)

Qué decir:

Durante toda la presentación usamos términos precisos: embeddings, vector store, similitud semántica, temperatura, chunking, overlap, retriever, system prompt, endpoint. No son términos de adorno, son los componentes reales del sistema.

Las decisiones están respaldadas por evidencia concreta. La temperatura 0.1 no es arbitraria: es la configuración que minimiza variabilidad sin colapsar la fluidez del lenguaje. El chunk size de 500 caracteres no es aleatorio: captura secciones temáticas completas del archivo de inventario sin fragmentar información relevante. El k=4 en el retriever equilibra contexto suficiente con no saturar la ventana del prompt.

Todo lo que afirmamos en el informe y en esta presentación está respaldado por el código que se puede ver, la base de conocimiento que se puede leer, y los resultados de las pruebas que se pueden reproducir.

Qué mostrar: Si queda tiempo, hacer una demo en vivo del endpoint de pedido con SKU ELEC-001 y stock 8, y mostrar que el sistema calcula la cantidad, identifica al proveedor, estima la fecha límite y activa el flag de aprobación gerencial.

Demo recomendada (orden sugerido)

1. Abrir el frontend en el navegador
 2. Consulta libre: "*¿Qué productos tienen stock crítico?*"
 3. Alerta: SKU ELEC-001
 4. Pedido: SKU ELEC-001, stock actual 8
 5. Prueba de robustez: consulta libre con "*¿Tienen stock de drones?*" — mostrar que el sistema reconoce que no tiene esa información
-

Datos concretos para mencionar en cualquier momento

- 6 SKUs gestionados en 4 categorías
- 2 productos en estado crítico al momento del análisis (ELEC-001: 8 vs mínimo 15 / ELEC-002: 3 vs mínimo 10)
- 5 proveedores con lead times entre 10 y 21 días
- Fragmentos de 500 caracteres con overlap de 50, top-4 recuperados por consulta
- Temperatura 0.1 en todos los endpoints
- Umbral de aprobación: \$5.000.000 CLP
- Buffer de seguridad: 20% sobre demanda proyectada a 30 días
- Modelo generativo: GPT-4o-mini vía GitHub Models (Azure Inference)
- Modelo de embeddings: text-embedding-3-small